

Strict Sourcing and Boundary Enforcement: Specifications in this template are restricted strictly to documented runtime libraries, modules, and developer tooling. Excluded future extensions (Nutrition tracking, BMI calculation, Wearable device integration, Workout reminders, and Fitness analytics dashboards) have no dependencies registered in this stack.

1. EXECUTION ENVIRONMENT & HARDWARE/SOFTWARE PREREQUISITES

Hardware and software baseline parameters required to initialize, develop, and execute the FitTrack AI backend service.

MINIMUM EXECUTION HARDWARE

- CPU:** Not specified in the source document
- System Memory (RAM):** Not specified in the source document
- Storage Space:** Not specified in the source document
- Network Interface:** TCP/IP network access for MongoDB connection and HTTPS egress to Google Gemini API

RECOMMENDED EXECUTION HARDWARE

- CPU:** Not specified in the source document
- System Memory (RAM):** Not specified in the source document
- Storage Space:** Not specified in the source document
- Network Bandwidth:** Low-latency broadband for API testing and third-party AI streaming

CORE SOFTWARE ENVIRONMENT

- Runtime Engine:** Node.js (version 16 or higher)
- Package Manager:** npm (bundled with Node.js)
- Operating System:** Cross-platform (Windows, macOS, Linux supported by Node.js runtime)
- Code Editor / IDE:** Visual Studio Code

TESTING & TOOLING ENVIRONMENT

- API Verification Clients:** Postman / ThunderClient
- Database Host:** MongoDB (local daemon or cloud connection URI)
- API Gateway Security:** CORS & JSON body parser middleware
- Dev Process Monitor:** nodemon (hot-reloading)

2. ARCHITECTURAL LAYER & TECHNOLOGY MAPPING

Layer-by-layer mapping of all documented software technologies, frameworks, and utility libraries.

ARCHITECTURAL LAYER	TECHNOLOGY / TOOL	PRIMARY PURPOSE	SPECIFIC USAGE IN FITTRACK AI
Runtime Layer	Node.js (v16+)	Asynchronous Event-Driven JavaScript Runtime	Executes backend server process, orchestrates non-blocking I/O event loops, and runs asynchronous database/AI network requests.
Application Framework	Express.js	Web Application Framework	Provides HTTP server scaffolding, route declaration, error handling pipeline, and REST controller binding.
Database Layer	MongoDB	NoSQL Document Database	Stores persistent JSON-like BSON documents for the User and Workout domain entities.
Object Modeling Layer	Mongoose	Object Data Modeling (ODM) Library	Enforces strict schema validation, type constraints, default values, and relational references (user Id in Workout model).
Security Layer (Hashing)	bcryptjs	Password Hashing & Salt Generation	Hashes user passwords during registration and compares plaintext inputs against persisted bcrypt hashes during login.

3. ARCHITECTURAL LAYER & TECHNOLOGY MAPPING (CONT.)

ARCHITECTURAL LAYER	TECHNOLOGY / TOOL	PRIMARY PURPOSE	SPECIFIC USAGE IN FITTRACK AI
Security Layer (Tokens)	jsonwebtoken (JWT)	Stateless Cryptographic Authentication	Issues digitally signed bearer tokens upon login/registration and authenticates private route access via Auth Guard middleware.
Middleware Layer	cors	Cross-Origin Resource Sharing	Enables controlled Cross-Origin HTTP requests between client applications (Postman/ThunderClient/Web) and Express endpoints.
Configuration Layer	dotenv	Zero-Dependency Environment Injector	Loads sensitive configurations from external .env file into runtime process .env object.
Development Tooling	nodemon	Development Process Monitor	Monitors codebase file mutations and restarts Node.js process automatically during iterative backend development.
Artificial Intelligence	Google Gemini API	Generative Foundation Model	Generates personalized exercise regimens (/api/ai/recommend) and historical workout performance insights (/api/ai/insights).
API Verification	Postman / ThunderClient	REST API Client Testing Suites	Used to execute three-tier sequential functional verification: 1. Auth APIs → 2. Workout APIs → 3. AI APIs.
Authoring Environment	Visual Studio Code	Integrated Development Environment	Primary IDE for source code development, workspace modularization, and script execution.

4. ENVIRONMENT VARIABLE DEFINITIONS (.ENV)

Centralized environmental configuration parameters loaded at runtime via dotenv.

VARIABLE NAME	DATA FORMAT	CONFIDENTIALITY	OPERATIONAL DEFINITION & ARCHITECTURAL PURPOSE
PORT	Integer	INTERNAL	Designates the TCP port on which the Express HTTP web server binds and listens for incoming client connections (e.g., 5000).
MONGO_URI	URI String	CONFIDENTIAL	Complete MongoDB connection URI string (including protocol, host, port, database name, and credentials) consumed by Mongoose.
JWT_SECRET	String Key	CONFIDENTIAL	Cryptographic secret key used by jsonwebtoken to digitally sign and cryptographically verify all issued Bearer JWT tokens.
GEMINI_API_KEY	Alphanumeric Key	CONFIDENTIAL	Authentication API key used to authorize outbound HTTPS requests from the server to the Google Gemini AI generative service.

5. DOCUMENTED TECHNOLOGY SELECTION RATIONALE

NODE.JS + EXPRESS.JS + MONGOOSE

Enables a unified JavaScript runtime spanning the entire application stack. Non-blocking event-driven I/O provides high-throughput processing for concurrent workout logging and queries, while Mongoose provides schema validation over MongoDB.

BCRYPTJS + JSONWEBTOKEN + GEMINI API

Delivers zero-state session scalability through cryptographic JWTs and one-way salted password hashing via bcryptjs. Direct integration with Google Gemini API enables intelligent generative workout recommendations and history-driven analytics.